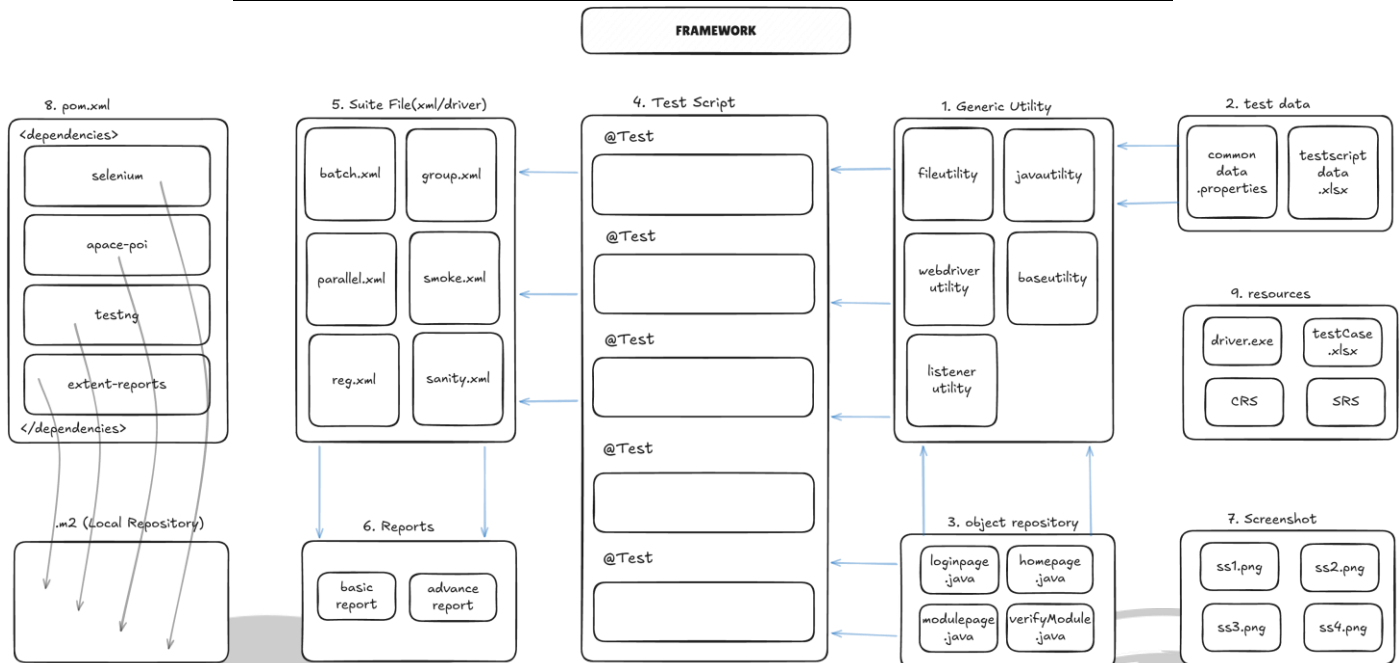


FRAMEWORK EXPLANATION



1. Generic Utility.
2. Test data
3. Object Repository
4. Test Scripts.
5. Test Suite
6. HTML Report.
7. Screenshots.
8. pom.xml
9. Resources

Definition

- **Framework:** A framework is a well-organized structure of reusable components where one driver file will take care of the execution without any manual intervention.

"In my current project, I have built a Selenium automation framework using Java, TestNG, Maven, and POM. The main focus while designing it was reusability, maintainability, and easy integration with CI/CD."

- We started with a **Generic Utility package**, where we keep all the reusable components. For example:
 - A Base Class that manages browser setup, teardown, and TestNG configuration annotations. All our test classes extend this, so setup is never repeated.
 - A Java Utility with helper methods like generating random numbers, getting system dates, etc.
 - A Properties File Utility and Excel Utility, which help us fetch data from external sources. This way, nothing is hardcoded in scripts.
 - A WebDriver Utility, where we handle browser-specific actions like waits, switching windows, handling alerts, dropdowns, etc.

- All the common **data**, like URLs, browser names, and credentials, are stored in a properties file. Test script data for scripts comes from Excel. This makes the framework flexible and data-driven.
- For the **Object Repository**, we use the Page Object Model. Each page of the application has its own class, which stores elements and their actions. This makes the framework maintainable — especially in Agile projects where UI changes are frequent. If a locator changes, I just update it in the page class and all scripts continue to work.
- The **Test Script** is written using TestNG classes. We use features like grouping, prioritization, parallel execution, and cross-browser execution. **Suite files** are created for different types of runs — like smoke, regression, or cross-browser testing.
- For **reporting**, we mainly rely on TestNG reports, and we also integrated Allure reports in Jenkins for better visualization. We capture **screenshots** of failed cases with the help of Listeners, which makes debugging much easier.
- The project is managed using Maven, so dependencies are handled in the **pom.xml** file. Maven also helps in integrating with Jenkins for CI/CD, where test suites are triggered automatically on every new build.
- Finally, we maintain **resources** like driver executables, configuration files, and documentation inside the resources folder. This way, the framework remains organized and easy for new team members to understand.

Overall, this framework is quite scalable and supports continuous testing. It reduces maintenance effort, handles frequent UI changes, and integrates well with the Agile process we follow.”

Framework Advantages

1. **Saves cost & effort** – Less rework, more return on investment.
2. **Covers more tests** – Increases efficiency and test coverage.
3. **Run on any environment** – Works on Testing, UAT, SIT, etc.
4. **Fast test writing** – Reusable parts make scripts quicker to build.
5. **Easy to maintain** – Data & elements stored in one place (POM + external files).
6. **Test anywhere** – Cross-browser, distributed, and agile-friendly.
7. **Auto screenshots** – Captures failures for quick debugging.
8. **Clear reports** – Gives structured and accurate results.
9. **Handy utilities** – Excel, Base class, Listeners, etc. ready to use.
10. **Reusable scripts** – Same test can work across builds.

Types of Frameworks

1. Data driven Framework
2. Modular driven framework
3. Test driven development framework
4. Hybrid Driven framework
5. Keyword Driven framework
6. Behaviour driven development framework
7. (BDD)

1. Data-Driven Framework (DDF)

- **Includes 9 components:**
 1. Generic Utility
 2. Object Repository
 3. Test Data
 4. Test Script
 5. Resources
 6. Suite File
 7. Screenshot
 8. HTML Reports
 9. pom.xml

Key Features

- Simplifies test data maintenance.
- Uses `@DataProvider` annotation.
- One test script can run with multiple data sets.
- Each test script has its own Data-Provider or Excel sheet.

When to Use?

- When large amounts of data need to be tested.
- Examples: E-commerce, Banking, Booking applications.

Pros

- Reduces the number of test scripts.
- Allows testing multiple scenarios with minimal code.

Cons

- Requires skilled testers, proficient in programming to design and maintain the framework.

2. Modular-Driven Framework (MDF)

When to Use?

- For large applications with multiple modules (e.g., CRM systems).

Key Features

- Application is divided into small modules.
- Test scripts are created for each module independently.

Pros

- Easier to manage and scale.
- Independent test scripts for each module.

Cons

- Maintaining consolidated reports for scripts is time-consuming.
- Not preferred in modern agile terms.

3. Keyword-Driven Framework (KDF)

When to Use?

- When manual testers need to write automation scripts with minimal coding knowledge.
- Best for small applications (below 100 test cases).

Key Features

- Test cases are written in Excel.
- Keywords map to key generic libraries (e.g., `openBrowser()`, `ClickElement()`, `EnterText()`).
- Not Suitable for **large** CRM-type applications bcz it **slow down** execution for **thousands of test cases**.

TC_ID	StepDescription	Keyword	LocatorType	LocatorValue	TestData
TC_01	Open URL	OpenBrowser()	URL	https://example.com	
TC_01	Enter username	EnterText()	id	username	John
TC_01	Enter password	EnterText()	id	password	12345
TC_01	Click login button	ClickElement()	xpath	//button[@id='login']	
TC_01	Close browser	CloseBrowser()			

```
public class Keywords {
```

```
    WebDriver driver;
```

```
    public void openBrowser(String url) {
```

```
        driver = new ChromeDriver();
```

```
        driver.manage().window().maximize();
```

```
        driver.get(url);
```

```
    }
```

```
    public void enterText(String locatorType, String locatorValue, String testData) {
```

```
        getElement(locatorType, locatorValue).sendKeys(testData);
```

```
    }
```

```
public void clickElement(String locatorType, String locatorValue) {  
    getElement(locatorType, locatorValue).click();  
}  
  
public void closeBrowser() {  
    driver.quit();  
}  
  
private WebElement getElement(String locatorType, String locatorValue) {  
    switch(locatorType.toLowerCase()) {  
        case "id": return driver.findElement(By.id(locatorValue));  
        case "name": return driver.findElement(By.name(locatorValue));  
        case "xpath": return driver.findElement(By.xpath(locatorValue));  
        case "css": return driver.findElement(By.cssSelector(locatorValue));  
        default: throw new RuntimeException("Invalid locator type: " + locatorType);  
    }  
}  
}
```

```
public class TestScript {
```

```
    public static void main(String[] args) throws Exception {  
        FileInputStream fis = new FileInputStream("TestSteps.xlsx");  
        Workbook workbook = WorkbookFactory.create(fis);  
        Sheet sheet = workbook.getSheet("sheetName");  
        Keywords keywords = new Keywords();  
        for (int i = 1; i <= sheet.getLastRowNum(); i++) {  
            Row row = sheet.getRow(i);  
            String keyword = row.getCell(2).getStringCellValue();
```

```
String locatorType = row.getCell(3).getStringCellValue();
```

```
String locatorValue = row.getCell(4).getStringCellValue();
```

```
String testData = row.getCell(5).getStringCellValue();
```

```
switch(keyword) {
```

```
    case "OpenBrowser": keywords.openBrowser(locatorValue); break;
```

```
    case "EnterText": keywords.enterText(locatorType, locatorValue, testData); break;
```

```
    case "ClickElement": keywords.clickElement(locatorType, locatorValue); break;
```

```
    case "CloseBrowser": keywords.closeBrowser(); break;
```

```
    default: System.out.println("Invalid keyword: " + keyword);
```

```
}
```

```
}
```

```
workbook.close();
```

```
}
```

```
}
```

Pros

- Easy maintenance and reusability.
- Non-coders can contribute via Excel.
- Makes automation highly modular and flexible.
- Clear separation between **test logic**, **test data**, and **test script**.

Cons

- **Slower execution** due to reading from external files.
- **Harder debugging** if keywords or mapping logic fails.
- Requires good **framework design discipline**, else it becomes a mess of keywords.
- Poor performance with **large datasets**.

Framework Development Stages

Automation frameworks are not built all at once—they go through **3 main stages**. Think of it like building a house: first you plan, then you build, and finally you live in it and use it!

1. Framework Design & Implementation (Planning and Setup Phase)

When? – Usually done in **Sprint 1** (the first few days/weeks of the project)

Who? – Experienced Automation Test Engineers (7 – 9 years)

What happens here?

- Decide **which tools** to use (like Selenium, TestNG, Maven, Jenkins).
- Create basic **utility classes** (e.g., for browser handling, data reading).
- Set up the **project structure** and folders.
- Create a small working model (called **POC – Proof of Concept**) for important features.
- Show this POC to the client or manager to get their approval.

Goal: Build the base or skeleton of the framework.

2. Framework Test Development (Building Phase)

When? – Usually in **Sprint 2** and continues through development

Who? – Automation Test Engineers (3 – 5 years)

What happens here?

- Start writing **real automation scripts** using the framework.
- Convert **manual test cases** into automation code.
- Store and organize all scripts in version control tools like **GitHub**.
- Review the code with teammates or seniors to ensure quality.

Goal: Fill your framework with working, reviewed, and reusable test scripts.

3. Framework Execution (Using and Monitoring Phase)

When? – Happens regularly after scripts are ready (during every release or sprint)

Who? – Mostly Senior Automation Test Engineers or DevOps team

What happens here?

- Test scripts are run using **Jenkins** or other CI tools (can be scheduled).
- After execution, an **HTML report** is generated showing pass/fail/skipped.
- If any script fails, screenshots are automatically captured.
- Engineers do **debugging**, **fix issues**, and **retest**.

Goal: Run the tests **smoothly**, find bugs, and provide reports to the team or client.

Summary:

Stage	Name	Main Tasks	Who Does It
1	Design & Implementation	Set up tools, create POC, write base classes	Senior Automation Engineers

Stage	Name	Main Tasks	Who Does It
2	Test Development	Write and review automation scripts	Automation Engineers
3	Execution	Run tests, report results, fix issues	Senior Testers or DevOps

